

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Licenciatura en Física

Documentación Técnica del Código

Laboratorio Virtual de Óptica: Motor Matemático y Módulos de Simulación

Propósito de este documento: presentar el código fuente completo de `utils_math.py` y de los cinco módulos de simulación en Streamlit, con explicaciones bloque por bloque y, donde corresponde, la referencia explícita a la ecuación o sección del reporte de titulación que cada fragmento implementa.

- `utils_math.py` — motor matemático compartido
- `1_Módulo_I_Reflexión_y_Refracción.py`
- `2_Módulo_II_Arreglo_de_Lentes.py`
- `3_Módulo_III_Interferencia.py`
- `4_Módulo_IV_Difracción.py`
- `5_Módulo_V_Polarización.py`

Autor:

Oscar Alejandro Godínez Deloya

Asesor:

Dr. Cristian Adán Mojica Casique

Ciudad de México, 2026

Índice

1	Introducción	2
2	utils_math.py: Motor Matemático Compartido	3
2.1	Bloque: Reflexión, Refracción y Fermat (usado por el Módulo I)	3
2.2	Bloque: Óptica Geométrica / Formalismo ABCD (usado por el Módulo II)	6
2.3	Bloque: Interferencia (usado por el Módulo III)	8
2.4	Bloque: Difracción de Fraunhofer (usado por el Módulo IV)	10
2.5	Bloque: Polarización y Formalismo de Jones (usado por el Módulo V)	11
3	1_Módulo_I_Reflexión_y_Refracción.py	14
4	2_Módulo_II_Arreglo_de_Lentes.py	19
5	3_Módulo_III_Interferencia.py	23
6	4_Módulo_IV_Difracción.py	26
7	5_Módulo_V_Polarización.py	28
8	Tabla Resumen: Vínculos y Notas de Congruencia	31

1 Introducción

Este documento acompaña al reporte de titulación *Desarrollo e Implementación de Material Didáctico Multimedia para el Fortalecimiento de la Enseñanza de la Óptica en la Licenciatura en Física* y tiene como propósito exclusivo servir de referencia técnica: explica qué hace cada bloque de código del laboratorio virtual y, cuando corresponde, señala la ecuación o sección del reporte que dicho bloque implementa numéricamente.

La organización sigue la arquitectura real del proyecto: primero se documenta `utils_math.py` (el motor matemático compartido por los cinco módulos), agrupado por el módulo de simulación al que sirve; después se documenta cada aplicación de Streamlit (`1_Módulo_I...py` a `5_Módulo_V...py`), bloque por bloque, en el mismo orden en que aparecen en el archivo.

Cada fragmento de código se acompaña de dos tipos de recuadro:

- **Explicación:** qué hace el bloque, línea por línea o agrupado cuando las líneas son repetitivas.
- **Vínculo con el reporte:** la ecuación, sección o apéndice del reporte que el bloque traduce a código. Cuando existe una discrepancia notable entre lo que dice el reporte y lo que hace el código, se señala en un recuadro adicional de **Nota de congruencia**.

2 utils_math.py: Motor Matemático Compartido

Este archivo concentra todas las funciones de cálculo puro (sin interfaz gráfica) que después importan los cinco módulos de Streamlit. Se organiza aquí en el mismo orden en que aparece en el archivo original, agrupado por el módulo de simulación al que sirve cada bloque.

2.1 Bloque: Reflexión, Refracción y Fermat (usado por el Módulo I)

```

1 import numpy as np
2
3 # =====
4 # MÓDULO II: REFLEXIÓN, REFRACCIÓN Y FRESNEL
5 # =====
6
7 def calcular_snell(n1, n2, theta_i_deg):
8     """
9     Calcula el ángulo de refracción utilizando la Ley de Snell.
10    Retorna el ángulo refractado en grados, un booleano para TIR,
11    y el ángulo crítico (si existe).
12    """
13    theta_i_rad = np.radians(theta_i_deg)
14    sin_theta_t = (n1 / n2) * np.sin(theta_i_rad)
15
16    # Cálculo del ángulo crítico (solo de medio denso a menos denso)
17    theta_critico = np.degrees(np.arcsin(n2 / n1)) if n1 > n2 else None
18
19    # Verificación de Reflexión Total Interna (TIR)
20    if sin_theta_t > 1.0:
21        return None, True, theta_critico
22
23    theta_t_deg = np.degrees(np.arcsin(sin_theta_t))
24    return theta_t_deg, False, theta_critico

```

Explicación

- **L.9:** convierte el ángulo de incidencia de grados (como lo entrega el deslizador de Streamlit) a radianes, que es lo que `numpy` espera en sus funciones trigonométricas.
- **L.10:** calcula $\sin \theta_t$ despejando directamente de la Ley de Snell: $\sin \theta_t = (n_1/n_2) \sin \theta_i$. Nótese que *no* se calcula θ_t todavía — solo su seno, porque ese valor es el que decide si existe o no una solución real.
- **L.13:** calcula el ángulo crítico $\theta_c = \arcsin(n_2/n_1)$ *únicamente* cuando $n_1 > n_2$ (medio denso a menos denso), ya que en caso contrario no existe ángulo crítico y la variable se deja como `None`.
- **L.16–17:** si $\sin \theta_t > 1$, no existe ángulo real de transmisión: es la condición matemática de Reflexión Total Interna. La función retorna `None` para el ángulo transmitido y `True` para la bandera de TIR.
- **L.19–20:** caso normal (sin TIR): se obtiene θ_t aplicando `arcsin` y se convierte de vuelta a grados para mostrarlo en la interfaz.

Vínculo con el reporte

Implementa directamente la Ley de Snell del reporte (ecuación $n_1 \sin \theta_i = n_2 \sin \theta_t$) y la condición de ángulo crítico (actualmente comentada en el cuerpo del reporte, pero activa y funcional aquí en el código). La lógica de `if sin_theta_t > 1.0` es la contraparte numérica exacta de la observación teórica del reporte: “el término... sobrepasa la unidad. En este régimen, el fenómeno de refracción cesa por completo”.

```

1 def calcular_fresnel(n1, n2, theta_i_deg):
2     """
3     Calcula los coeficientes de reflectancia (R) y transmitancia (T)

```

```

4  para las componentes s, p, y el promedio para luz no polarizada.
5  Recibe el ángulo de incidencia en grados.
6  """
7  theta_i_rad = np.radians(theta_i_deg)
8  sin_theta_t = (n1 / n2) * np.sin(theta_i_rad)
9
10 # Caso 1: Reflexión Total Interna (TIR)
11 if sin_theta_t >= 1.0:
12     return 1.0, 1.0, 1.0, 0.0, 0.0, 0.0
13
14 theta_t_rad = np.arcsin(sin_theta_t)
15
16 # Caso 2: Incidencia normal estricta para evitar indeterminaciones 0/0
17 if theta_i_deg == 0.0:
18     R_normal = ((n1 - n2) / (n1 + n2))**2
19     return R_normal, R_normal, R_normal, 1.0 - R_normal, 1.0 - R_normal, 1.0 - R_normal
20
21 cos_i = np.cos(theta_i_rad)
22 cos_t = np.cos(theta_t_rad)
23
24 # Coeficientes de reflexión en amplitud y potencia (Polarización s)
25 r_s = (n1 * cos_i - n2 * cos_t) / (n1 * cos_i + n2 * cos_t)
26 R_s = r_s**2
27 T_s = 1.0 - R_s
28
29 # Coeficientes de reflexión en amplitud y potencia (Polarización p)
30 r_p = (n1 * cos_t - n2 * cos_i) / (n1 * cos_t + n2 * cos_i)
31 R_p = r_p**2
32 T_p = 1.0 - R_p
33
34 # Promedios para luz no polarizada (útil para el Módulo II general)
35 R_prom = (R_s + R_p) / 2.0
36 T_prom = 1.0 - R_prom
37
38 return R_s, R_p, R_prom, T_s, T_p, T_prom

```

Explicación

- **L.9–10:** si hay TIR, toda la energía se refleja: $R = 1$, $T = 0$ para ambas polarizaciones y el promedio, sin necesidad de evaluar las fórmulas de Fresnel (que en ese régimen involucrarían raíces de números negativos).
- **L.15–17:** caso especial de incidencia normal ($\theta_i = 0$): las fórmulas generales de Fresnel dividen por $\cos\theta_i - \cos\theta_t$ que tiende a $0/0$ en ese límite, así que se usa directamente la fórmula asintótica $R = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2$.
- **L.22–23:** se calculan los cosenos de incidencia y transmisión, insumos de las fórmulas generales de Fresnel.
- **L.26–28:** coeficiente de amplitud r_s y reflectancia $R_s = r_s^2$ para la polarización perpendicular (s); $T_s = 1 - R_s$ por conservación de energía.
- **L.31–33:** lo mismo para la polarización paralela (p).
- **L.36–37:** promedio de ambas polarizaciones, útil para luz natural (no polarizada), que es lo que se reporta como “Reflectancia (R)” principal en la interfaz.

Vínculo con el reporte

Corresponde exactamente a los coeficientes de Fresnel del Apéndice C del reporte, incluyendo la identidad de conservación de energía $R + T = 1$, que aquí se impone *por construcción* en cada rama ($T_s = 1.0 - R_s$), en vez de verificarse a posteriori.

```

1 def analizar_camino_optico(n1, n2, x_A, y_A, x_B, y_B, x_interfaz, resolucion=200):
2     """

```

```

3  Calcula el Camino Óptico (OPL) puntual para un x_interfaz arbitrario
4  y genera la curva continua para demostrar el principio variacional de Fermat.
5  """
6  L1_puntual = np.sqrt((x_interfaz - x_A)**2 + y_A**2)
7  L2_puntual = np.sqrt((x_B - x_interfaz)**2 + y_B**2)
8  opl_puntual = n1 * L1_puntual + n2 * L2_puntual
9
10 x_array = np.linspace(x_A - 2, x_B + 2, resolucion)
11 L1_array = np.sqrt((x_array - x_A)**2 + y_A**2)
12 L2_array = np.sqrt((x_B - x_array)**2 + y_B**2)
13 opl_array = n1 * L1_array + n2 * L2_array
14
15 indice_min = np.argmin(opl_array)
16 x_min = x_array[indice_min]
17 opl_min = opl_array[indice_min]
18
19 return opl_puntual, x_array, opl_array, x_min, opl_min

```

Explicación

- **L.6–8:** calcula el Camino Óptico para el punto de cruce *actual* elegido por el usuario en el deslizador: distancia de *A* al punto de cruce (ponderada por n_1) más distancia del cruce a *B* (ponderada por n_2).
- **L.10:** genera un arreglo de 200 posibles puntos de cruce x a lo largo de la interfaz, para poder graficar la curva completa del “pozo de potencial” del OPL.
- **L.11–13:** evalúa el OPL para *cada* uno de esos 200 puntos simultáneamente (vectorizado con `numpy`), sin necesidad de un ciclo `for` explícito.
- **L.15–17:** localiza el índice del mínimo absoluto de la curva con `np.argmin`, y extrae tanto la coordenada x como el valor de OPL en ese mínimo. Este es el paso que *numéricamente* encuentra la trayectoria real que predice el Principio de Fermat.

Vínculo con el reporte

Esta función es la traducción directa a código de la ecuación del reporte, $LCO(x) = n_{AP}\overline{AP} + n_{BP}\overline{PB}$, y de la condición de extremo de la ecuación. En vez de resolver $d(LCO)/dx = 0$ analíticamente, el código encuentra el mínimo por búsqueda numérica sobre una malla de 200 puntos — exactamente lo que describe el Apéndice A cuando dice que la “discretización de malla” introduce una pequeña discrepancia respecto al mínimo teórico exacto.

Nota de congruencia

La interfaz (`1_Módulo_I...py`) etiqueta esta cantidad como **OPL** en todos los textos y métricas mostradas al usuario. El reporte, en cambio, la llama **LCO** (Longitud de Camino Óptico) en el cuerpo principal. Es la misma cantidad física con dos nombres distintos entre el código y el texto — conviene unificar la nomenclatura en el reporte para que coincida con lo que el estudiante ve en pantalla.

2.2 Bloque: Óptica Geométrica / Formalismo ABCD (usado por el Módulo II)

```

1 # =====
2 # MÓDULO I: ÓPTICA GEOMÉTRICA (MATRICES ABCD)
3 # =====
4
5 def matriz_translacion(d):
6     """Matriz de propagación en el espacio libre por una distancia d."""
7     return np.array([[1.0, float(d)], [0.0, 1.0]])
8
9 def matriz_lente_delgada(f):
10    """Matriz de transformación para una lente delgada de distancia focal f."""
11    if f == 0:
12        f = 1e-9 # Pequeña perturbación para evitar singularidad numérica
13    return np.array([[1.0, 0.0], [-1.0 / float(f), 1.0]])
14
15 def matriz_espejo_esferico(R):
16    """Matriz de transformación para un espejo esférico de radio R."""
17    if R == 0:
18        R = 1e-9
19    return np.array([[1.0, 0.0], [-2.0 / float(R), 1.0]])

```

Explicación

- **L.5–7:** matriz de traslación en espacio libre: matriz 2×2 con 1 en la diagonal y d en la esquina superior derecha. Al multiplicarla por el vector de rayo $(y, \alpha)^T$, deja α intacto y suma $d \cdot \alpha$ a la altura y .
- **L.9–13:** matriz de lente delgada de distancia focal f . La línea 11 evita una división entre cero: si el usuario pone $f = 0$ (lente sin sentido físico), se sustituye por un número casi cero pero no exactamente cero, para que numpy no arroje inf o nan.
- **L.15–18:** misma idea, pero para un espejo esférico de radio R : el elemento (2,1) de la matriz es $-2/R$ en vez de $-1/f$.

Vínculo con el reporte

`matriz_translacion` es la ecuación del reporte, $M_{\text{tras}}(d) = \begin{pmatrix} 1 & d/n \\ 0 & 1 \end{pmatrix}$, evaluada aquí para $n = 1$ (aire) — el código no generaliza a medios con $n \neq 1$. `matriz_lente_delgada` corresponde a la ecuación $M_{\text{lente}} = \begin{pmatrix} 1 & 0 \\ -1/f & 1 \end{pmatrix}$. `matriz_espejo_esferico` corresponde a la ecuación del Apéndice D, $M_{\text{espejo}}(R) = \begin{pmatrix} 1 & 0 \\ -2/R & 1 \end{pmatrix}$.

```

1 def trazar_rayo_sistema(y0, theta0_deg, componentes, d_final=20.0):
2     """Traza las coordenadas transversales de un rayo a lo largo de un eje óptico desplegado."""
3     theta0_rad = np.radians(theta0_deg)
4     rayo_actual = np.array([[y0], [theta0_rad]])
5
6     posiciones_z = [0.0]
7     alturas_y = [y0]
8     z_actual = 0.0
9
10    for tipo, valor, d in componentes:
11        if d > 0:
12            M_trans = matriz_translacion(d)
13            rayo_actual = np.dot(M_trans, rayo_actual)
14            z_actual += d
15            posiciones_z.append(z_actual)
16            alturas_y.append(rayo_actual[0, 0])
17
18        if tipo == 'lente':
19            M_comp = matriz_lente_delgada(valor)
20        elif tipo == 'espejo':
21            M_comp = matriz_espejo_esferico(valor)

```

```

22     else:
23         continue
24
25     rayo_actual = np.dot(M_comp, rayo_actual)
26
27     if len(componentes) > 0:
28         M_trans_final = matriz_translacion(d_final)
29         rayo_actual = np.dot(M_trans_final, rayo_actual)
30         z_actual += d_final
31         posiciones_z.append(z_actual)
32         alturas_y.append(rayo_actual[0, 0])
33
34     return posiciones_z, alturas_y, rayo_actual

```

Explicación

- **L.3–4:** construye el vector de rayo inicial $\vec{r}_0 = (y_0, \theta_0)^T$, convirtiendo el ángulo a radianes.
- **L.6–8:** listas donde se va acumulando la posición axial z y la altura y del rayo en cada punto de interés, para poder graficar la trayectoria completa después.
- **L.10:** recorre cada componente óptico del banco (una tupla (tipo, valor, distancia_al_anterior)).
- **L.11–16:** si hay una distancia de separación d antes del componente, primero se propaga el rayo en espacio libre multiplicando por $M_{\text{tras}}(d)$, y se registra la nueva posición para la gráfica.
- **L.18–22:** según el tipo de componente ('lente' o 'espejo'), se construye la matriz correspondiente; si el tipo no se reconoce, se salta al siguiente componente sin aplicar ninguna transformación.
- **L.24:** aplica la matriz del componente óptico al vector de rayo (`np.dot` es el producto matriz-vector).
- **L.26–30:** al terminar de recorrer todos los componentes, se propaga el rayo una distancia adicional `d_final` (la "pantalla de proyección"), para poder visualizar dónde termina el rayo después del último elemento.

Vínculo con el reporte

Implementa la ecuación del reporte, $\vec{r}_{\text{final}} = M_N \cdots M_2 M_1 \vec{r}_0$, aplicando las matrices *en el orden físico real* en que el haz las atraviesa (multiplicación acumulada de izquierda a derecha sobre el vector, equivalente a multiplicar las matrices de derecha a izquierda como indica el reporte).

Nota de congruencia

El vector de rayo del reporte usa el ángulo *reducido* $n\alpha$ (ecuación $\vec{r} = (y, n\alpha)^T$), mientras que el código usa el ángulo geométrico puro sin multiplicar por n (`theta0_rad`, sin factor de índice de refracción). Esto es consistente porque todo el sistema se modela en aire ($n = 1$) en ambos casos, por lo que $n\alpha = \alpha$ numéricamente — pero si en algún momento se generaliza el simulador a medios con $n \neq 1$, este detalle tendría que revisarse.

```

1 def obtener_matriz_sistema(componentes):
2     """Calcula la composición multiplicativa analítica del sistema completo."""
3     M_acumulada = np.eye(2)
4     for tipo, valor, d in componentes:
5         if d > 0:
6             M_acumulada = np.dot(matriz_translacion(d), M_acumulada)
7         if tipo == 'lente':
8             M_comp = matriz_lente_delgada(valor)
9         elif tipo == 'espejo':
10            M_comp = matriz_espejo_esferico(valor)
11        else:
12            continue
13        M_acumulada = np.dot(M_comp, M_acumulada)

```



```
14 return M_acumulada
```

Explicación

- **L.3:** inicializa la matriz acumulada como la identidad 2×2 (`np.eye(2)`) — el elemento neutro de la multiplicación matricial.
- **L.4–11:** recorre cada componente y va multiplicando *por la izquierda* la matriz de traslación y luego la del componente óptico, exactamente en el orden $M_N \cdots M_2 M_1$ que exige la composición correcta de sistemas ópticos.
- **L.12:** retorna la matriz M_{sys} completa del sistema, de la cual la interfaz extrae A , B , C , D .

Vínculo con el reporte

Corresponde a la matriz equivalente del sistema, ecuación del reporte, $M_{sys} = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$. El elemento C que retorna esta función es el que la interfaz usa para calcular $f_{eq} = -1/C$ y para detectar sistemas afocales.

2.3 Bloque: Interferencia (usado por el Módulo III)

```
1 # =====
2 # MÓDULO III: INTERFERENCIA
3 # =====
4
5 def calcular_patron_young(longitud_onda_nm, d_mm, L_cm, tamano_pantalla_cm, resolucion=500):
6     """Calcula el patrón de irradiancia bidimensional exacto no paraxial para Young."""
7     lam = float(longitud_onda_nm) * 1e-9
8     d = float(d_mm) * 1e-3
9     L = float(L_cm) * 1e-2
10    tamano = float(tamano_pantalla_cm) * 1e-2
11
12    x = np.linspace(-tamano/2, tamano/2, resolucion)
13    y = np.linspace(-tamano/2, tamano/2, resolucion)
14    X, Y = np.meshgrid(x, y)
15
16    k = 2.0 * np.pi / lam
17    r1 = np.sqrt(X**2 + (Y - d/2)**2 + L**2)
18    r2 = np.sqrt(X**2 + (Y + d/2)**2 + L**2)
19
20    delta_phi = k * (r2 - r1)
21    intensidad = np.cos(delta_phi / 2.0)**2
22    return X, Y, intensidad
```

Explicación

- **L.7–10:** convierte todos los parámetros de las unidades “amigables” de la interfaz (nm, mm, cm) al Sistema Internacional (metros), que es lo que usan las fórmulas físicas.
- **L.12–14:** construye una malla bidimensional X , Y de puntos sobre la pantalla de observación, usando `np.meshgrid` para evaluar la irradiancia en cada punto simultáneamente (vectorizado, sin ciclos `for`).
- **L.16:** calcula el número de onda $k = 2\pi/\lambda$.
- **L.17–18:** calcula las distancias *exactas* (tridimensionales, sin ninguna aproximación de ángulo pequeño) de cada rendija (S_1 en $y = d/2$, S_2 en $y = -d/2$) a cada punto (X, Y) de la pantalla, situada a una distancia axial L .
- **L.20:** calcula la diferencia de fase exacta $\delta = k(r_2 - r_1)$.
- **L.21:** evalúa la irradiancia normalizada $I/I_{\max} = \cos^2(\delta/2)$ en cada punto de la malla.

Vínculo con el reporte

Esta función es la implementación numérica *exacta* de la ecuación del reporte, $I = 4I_0 \cos^2(\delta/2)$ (normalizada aquí a $I_{\max} = 1$), combinada con la fase de la ecuación $\delta = k(r_2 - r_1)$. Es también la confirmación en código de la afirmación del reporte: “el motor de cálculo del Módulo III no usa esta aproximación [paraxial]: evalúa r_1 y r_2 de manera exacta sobre toda una malla bidimensional”, y respalda directamente la derivación completa del Apéndice A (espaciamiento hiperbólico exacto de las franjas de Young).

```

1 def calcular_patron_michelson(longitud_onda_nm, delta_d_um, f_lente_cm, tamaño_pantalla_cm,
2   resolución=500):
3     """Calcula las franjas de igual inclinación (anillos de Haidinger) de Michelson."""
4     lam = float(longitud_onda_nm) * 1e-9
5     delta_d = float(delta_d_um) * 1e-6
6     f = float(f_lente_cm) * 1e-2
7     tamaño = float(tamaño_pantalla_cm) * 1e-2
8
9     x = np.linspace(-tamaño/2, tamaño/2, resolución)
10    y = np.linspace(-tamaño/2, tamaño/2, resolución)
11    X, Y = np.meshgrid(x, y)
12
13    R = np.sqrt(X**2 + Y**2)
14    theta = np.arctan(R / f)
15
16    k = 2.0 * np.pi / lam
17    delta_phi = k * (2.0 * delta_d * np.cos(theta))
18    intensidad = np.cos(delta_phi / 2.0)**2
19    return X, Y, intensidad

```

Explicación

- **L.3–6:** conversión de unidades a metros: nótese que Δd viene en micrómetros (μm) desde la interfaz, ya que a esa escala son sensibles los desplazamientos del espejo.
- **L.11:** distancia radial $R = \sqrt{X^2 + Y^2}$ de cada punto de la pantalla al centro del patrón.
- **L.12:** ángulo de inclinación $\theta = \arctan(R/f)$ de cada rayo, donde f es la distancia focal de la lente proyectora — es lo que convierte la posición radial en pantalla a un ángulo físico de incidencia.
- **L.15:** diferencia de fase $\Delta\phi = \frac{4\pi}{\lambda} \Delta d \cos\theta$, evaluada para *cada* ángulo θ de la malla — es este término $\cos\theta$ el que hace que la fase dependa solo del radio y produzca los anillos concéntricos.
- **L.16:** irradiancia $I/I_{\max} = \cos^2(\Delta\phi/2)$, misma fórmula de superposición que en Young.

Vínculo con el reporte

Implementa la ecuación del reporte, $\delta = \frac{4\pi\Delta L}{\lambda}$, generalizada aquí con el factor $\cos\theta$ para producir los anillos de Haidinger descritos en la figura del interferómetro de Michelson y su pie de figura ($\delta = 4\pi\Delta L/\lambda$ con dependencia angular).

Nota de congruencia

La ecuación tal como está escrita en el cuerpo del reporte **no incluye** el factor $\cos\theta$ que sí aparece en el código (línea 15) y en el pie de la figura del interferómetro de Michelson en el reporte. Sin ese factor angular, la fase sería constante en toda la pantalla y no se formarían anillos concéntricos — es decir, la ecuación activa del cuerpo del reporte está incompleta respecto a lo que realmente calcula el simulador. Se recomienda añadir el $\cos\theta$ a la ecuación del cuerpo principal.

2.4 Bloque: Difracción de Fraunhofer (usado por el Módulo IV)

```

1 # =====
2 # MÓDULO IV: DIFRACCIÓN DE FRAUNHOFER
3 # =====
4
5 def calcular_difraccion_rendija(longitud_onda_nm, ancho_a_mm, L_cm, tamano_pantalla_cm, resolucion
   =500):
6     """Calcula la difracción de Fraunhofer para una rendija simple vertical."""
7     lam = float(longitud_onda_nm) * 1e-9
8     a = float(ancho_a_mm) * 1e-3
9     L = float(L_cm) * 1e-2
10    tamano = float(tamano_pantalla_cm) * 1e-2
11
12    x = np.linspace(-tamano/2, tamano/2, resolucion)
13    y = np.linspace(-tamano/2, tamano/2, resolucion)
14    X, Y = np.meshgrid(x, y)
15
16    distancia = np.sqrt(X**2 + Y**2 + L**2)
17    sin_theta_x = X / distancia
18
19    factor_x = a * sin_theta_x / lam
20    intensidad = np.sinc(factor_x)**2
21    return X, Y, intensidad

```

Explicación

- **L.7–10:** conversión de unidades a metros, igual que en los módulos anteriores.
- **L.12–14:** malla bidimensional de puntos de observación sobre la pantalla.
- **L.16:** distancia *exacta* (geométrica, tridimensional) de cada punto de la pantalla al centro de la apertura.
- **L.17:** en vez de usar la aproximación de campo lejano $\sin\theta \approx x/L$, se calcula el seno *geométrico exacto* $\sin\theta_x = X/\text{distancia}$, dividiendo entre la hipotenusa real y no solo entre L .
- **L.19:** variable auxiliar $\alpha/\pi = a \sin\theta_x/\lambda$ (el argumento normalizado que exige `np.sinc`).
- **L.20:** irradiancia $I/I_0 = \text{sinc}^2(\cdot)$, usando la función `numpy.sinc`, que ya trae incorporada la convención normalizada $\text{sinc}(u) = \sin(\pi u)/(\pi u)$.

Vínculo con el reporte

Corresponde a la ecuación del reporte, $I(\theta_m) = I_0 \text{sinc}^2(a \sin\theta_m/\lambda)$, con la convención normalizada de `numpy.sinc` que el reporte menciona explícitamente: “*coherente con la implementación computacional nativa de `numpy.sinc`*”.

Nota de congruencia

La interfaz (`4_Módulo_IV...py`) le muestra al estudiante la fórmula paraxial simplificada $\alpha = \pi ax/(\lambda L)$ (usando $\sin\theta \approx x/L$), pero el código de esta función — la que realmente alimenta la gráfica — usa el seno geométrico exacto de la línea 17 ($X/\text{distancia}$, no X/L). Para $L \gg a$ la diferencia es despreciable, pero la fórmula desplegada en pantalla no es, estrictamente, la que se está evaluando.

```

1 def calcular_difraccion_rectangular(longitud_onda_nm, ancho_a_mm, altura_b_mm, L_cm,
   tamano_pantalla_cm, resolucion=500):
2     """Calcula la distribución espacial exacta 2D para una apertura rectangular."""
3     lam = float(longitud_onda_nm) * 1e-9
4     a = float(ancho_a_mm) * 1e-3
5     b = float(altura_b_mm) * 1e-3
6     L = float(L_cm) * 1e-2

```

```

7     tamaño = float(tamaño_pantalla_cm) * 1e-2
8
9     x = np.linspace(-tamaño/2, tamaño/2, resolución)
10    y = np.linspace(-tamaño/2, tamaño/2, resolución)
11    X, Y = np.meshgrid(x, y)
12
13    distancia = np.sqrt(X**2 + Y**2 + L**2)
14    sin_theta_x = X / distancia
15    sin_theta_y = Y / distancia
16
17    factor_x = a * sin_theta_x / lam
18    factor_y = b * sin_theta_y / lam
19
20    intensidad = (np.sinc(factor_x)**2) * (np.sinc(factor_y)**2)
21    return X, Y, intensidad

```

Explicación

Misma lógica que la rendija simple, pero calculando *ambos* senos geométricos ($\sin\theta_x$ para el ancho a y $\sin\theta_y$ para el alto b de la apertura) y multiplicando los dos factores sinc^2 independientes (L.20), en vez de uno solo.

Vínculo con el reporte

Implementa la ecuación del reporte, $I(\theta_m, \theta_n) = I_0 \text{sinc}^2(a \sin\theta_m/\lambda) \text{sinc}^2(b \sin\theta_n/\lambda)$, consecuencia de la separabilidad de la transformada de Fourier en coordenadas cartesianas que el reporte discute justo antes de esa ecuación.

2.5 Bloque: Polarización y Formalismo de Jones (usado por el Módulo V)

```

1  # =====
2  # MÓDULO V: POLARIZACIÓN Y FORMALISMO DE JONES
3  # =====
4
5  def vector_jones_inicial(estado):
6      """Retorna el vector de Jones estructurado y normalizado."""
7      if estado == 'Lineal Horizontal':
8          return np.array([[1.0], [0.0]], dtype=complex)
9      elif estado == 'Lineal Vertical':
10         return np.array([[0.0], [1.0]], dtype=complex)
11     elif estado == 'Lineal +45°':
12         return np.array([[1.0/np.sqrt(2)], [1.0/np.sqrt(2)]], dtype=complex)
13     elif estado == 'Circular Derecha':
14         return np.array([[1.0/np.sqrt(2)], [-1j/np.sqrt(2)]], dtype=complex)
15     elif estado == 'Circular Izquierda':
16         return np.array([[1.0/np.sqrt(2)], [1j/np.sqrt(2)]], dtype=complex)
17     else:
18         return np.array([[1.0], [0.0]], dtype=complex)

```

Explicación

Cada rama retorna un vector columna complejo de 2×1 (`dtype=complex` permite usar $1j$, la unidad imaginaria de Python) representando un estado de polarización canónico: horizontal $(1,0)^T$, vertical $(0,1)^T$, lineal a 45° (ambas componentes reales e iguales), y circular derecha/izquierda, donde la diferencia de signo en la componente imaginaria ($-i$ vs. $+i$) codifica el sentido de giro (quiralidad) del campo.

Vínculo con el reporte

Corresponde a la ecuación del reporte, $\vec{J} = (E_{0x}e^{i\phi_x}, E_{0y}e^{i\phi_y})^T$, evaluada en los casos particulares que el reporte menciona explícitamente en el párrafo comentado “Vectores de Jones”: las polarizaciones lineales horizontal

$(1,0)^T$ y vertical $(0,1)^T$, y los estados circulares $\frac{1}{\sqrt{2}}(1,\mp i)^T$.

Nota de congruencia

El párrafo `%\paragraph{Vectores de Jones.}` que describe exactamente esta función está comentado (inactivo) en el reporte. El código confirma que la funcionalidad sí existe y está en uso; conviene descomentar ese párrafo o reescribirlo brevemente en el cuerpo activo del reporte.

```
1 def matriz_polarizador_lineal(theta_deg):
2     """Matriz de Jones para un polarizador lineal orientado a theta grados."""
3     th = np.radians(theta_deg)
4     c, s = np.cos(th), np.sin(th)
5     return np.array([[c**2, c*s], [c*s, s**2]], dtype=complex)
```

Explicación

Construye directamente la matriz 2×2 del polarizador lineal a partir de $\cos \theta$ y $\sin \theta$: los elementos diagonales son $\cos^2 \theta$ y $\sin^2 \theta$, y los elementos fuera de la diagonal son ambos $\cos \theta \sin \theta$ (la matriz es simétrica, como corresponde a un proyector ortogonal).

Vínculo con el reporte

Es exactamente la ecuación del reporte, $M_{\text{pol}}(\theta) = \begin{pmatrix} \cos^2 \theta & \cos \theta \sin \theta \\ \cos \theta \sin \theta & \sin^2 \theta \end{pmatrix}$.

```
1 def matriz_retardador(theta_deg, gamma_deg):
2     """Matriz de Jones para un retardador general con rotación espacial."""
3     th = np.radians(theta_deg)
4     gamma = np.radians(gamma_deg)
5     c, s = np.cos(th), np.sin(th)
6
7     R_inv = np.array([[c, -s], [s, c]], dtype=complex)
8     R = np.array([[c, s], [-s, c]], dtype=complex)
9
10    W = np.array([[np.exp(1j * gamma / 2.0), 0.0],
11                  [0.0, np.exp(-1j * gamma / 2.0)]], dtype=complex)
12
13    return np.dot(R_inv, np.dot(W, R))
```

Explicación

- **L.7–8:** construye las matrices de rotación $R(-\theta)$ (llamada `R_inv`) y $R(\theta)$ (llamada `R`), necesarias para expresar el retardador — definido originalmente en su propio sistema de ejes rápido/lento — en el sistema de referencia horizontal del laboratorio.
- **L.10–11:** matriz de fase pura $W(\Gamma) = \text{diag}(e^{i\Gamma/2}, e^{-i\Gamma/2})$ en el sistema propio del retardador.
- **L.13:** producto matricial $R(-\theta)W(\Gamma)R(\theta)$, evaluado de derecha a izquierda con dos llamadas anidadas a `np.dot`.

Vínculo con el reporte

Esta función es la implementación exacta, término por término, del **Anexo de Matrices de Jones: Rotación y Láminas Retardadoras** añadido al reporte: la matriz de rotación (ecuación $R(\theta)$), y la construcción general del retardador (ecuación $M_{\text{ret}}(\theta, \Gamma) = R(-\theta)W(\Gamma)R(\theta)$). Es una validación cruzada directa: el anexo que se redactó a partir de la necesidad teórica detectada coincide exactamente con lo que el código ya calculaba.

```

1 def curva_elipse_polarizacion(Ex, Ey, resolucion=300):
2     """Genera la trayectoria temporal del vector de campo eléctrico real."""
3     t = np.linspace(0, 2 * np.pi, resolucion)
4     Ex_t = np.real(complex(Ex) * np.exp(-1j * t))
5     Ey_t = np.real(complex(Ey) * np.exp(-1j * t))
6     return Ex_t, Ey_t

```

Explicación

Genera 300 instantes de tiempo t entre 0 y 2π , y para cada uno evalúa la parte real de $E_x(t) = \text{Re}[E_x e^{-i\omega t}]$ y $E_y(t) = \text{Re}[E_y e^{-i\omega t}]$ (con $\omega t \equiv t$ adimensional). El arreglo resultante traza la elipse completa que describe la punta del vector de campo eléctrico real a lo largo de un periodo.

Vínculo con el reporte

Corresponde a la fórmula de reconstrucción del campo real mencionada en el párrafo comentado del reporte: $\vec{E}(t) = \text{Re}[\vec{J}e^{-i\omega t}]$, y es la base geométrica de la elipse de polarización descrita en la ecuación de la elipse general y la figura de polarización del reporte.

```

1 def jones_a_euler_latex(Jx, Jy):
2     """Traduce un estado de Jones complejo a expresiones nativas en formato LaTeX."""
3     def formatear_componente(z):
4         amp = np.abs(z)
5         fase = np.angle(z)
6
7         if amp < 1e-4:
8             return "0"
9         if abs(fase) < 1e-4:
10            return f"{amp:.2f}"
11
12            return f"{amp:.2f} e^{i {fase:.2f}}"
13
14        # Forzar conversión a escalares complejos nativos para blindar la operación
15        comp_x = formatear_componente(complex(Jx))
16        comp_y = formatear_componente(complex(Jy))
17
18        return r"\vec{E}_{salida} = \begin{pmatrix} " + comp_x + r" \\" + comp_y + r" \end{pmatrix}"

```

Explicación

Función puramente de presentación (no física): convierte cada componente compleja $z = |z|e^{i\phi}$ a su representación en notación de Euler como cadena de texto LaTeX. Si la amplitud es prácticamente cero, escribe simplemente "0"; si la fase es prácticamente cero, omite el exponencial y solo muestra la amplitud; en el caso general, construye la cadena `amp e^{i fase}`. Al final concatena ambas componentes dentro de un `pmatrix` de LaTeX que Streamlit renderiza con `st.latex`.

Vínculo con el reporte

Implementa la afirmación del reporte: “El resultado analítico se expone en formato LaTeX utilizando la notación de Euler ($Ae^{i\phi}$)”, en la descripción del Módulo V (Capítulo 4).

3 1_Módulo_I_Reflexión_y_Refracción.py

```

1 import streamlit as st
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from utils_math import calcular_snell, analizar_camino_optico, calcular_fresnel
5
6 # Configuración de la página web
7 st.set_page_config(page_title="Módulo II: Reflexión y Refracción", layout="wide")
8
9 st.title("Módulo II: Reflexión y Refracción")
10 st.markdown("Estudio de la Ley de Snell, verificación del Principio de Fermat y aplicaciones en Guías de Onda.")

```

Explicación

Importa el motor matemático desde `utils_math` y configura la página: `layout="wide"` usa todo el ancho de la pantalla (necesario para las tres columnas de pestañas que vienen más adelante). `st.title` y `st.markdown` despliegan el encabezado visible al usuario.

Nota de congruencia

Tanto el título de la pestaña del navegador (`page_title`) como el título visible en pantalla (`st.title`) dicen “Módulo II”, mientras que el nombre del archivo y el reporte ya usan “Módulo I” para este contenido (Reflexión, Refracción y Fermat). Es la inconsistencia de mayor visibilidad detectada: si se presenta la aplicación en vivo, la pantalla dirá lo contrario de lo que dice el reporte y los cuadernillos. Conviene corregir estas dos líneas a “Módulo I: Reflexión y Refracción”.

```

1 # --- BARRA LATERAL (CONTROLES) ---
2 st.sidebar.header("Parámetros del Sistema (Snell)")
3 n1 = st.sidebar.slider("Índice de refracción (Medio 1 / Núcleo)", 1.0, 3.0, 1.5, 0.01)
4 n2 = st.sidebar.slider("Índice de refracción (Medio 2 / Revestimiento)", 1.0, 3.0, 1.3, 0.01)
5 theta_i_deg = st.sidebar.slider("Ángulo de Incidencia  $\theta_1$  (°)", 0.00, 89.99, 75.00, 0.01)
6 st.sidebar.divider()
7
8 st.sidebar.header("Principio de Fermat")
9 st.sidebar.markdown("Encuentra la trayectoria real minimizando el camino óptico entre A y B.")
10 x_interfaz = st.sidebar.slider("Desliza el punto de incidencia 'x' en la frontera",
11                               min_value=-5.0, max_value=5.0, value=0.0, step=0.1)

```

Explicación

Cada `st.sidebar.slider` crea un control deslizante en el panel lateral: los tres primeros parámetros (n_1 , n_2 , θ_i) alimentan la Pestaña 1 (Snell/Fresnel); el último (x_{interfaz}) alimenta la Pestaña 2 (Fermat), permitiendo al estudiante mover manualmente el punto de cruce en la interfaz dieléctrica.

Vínculo con el reporte

Los valores por omisión ($n_1 = 1.5$, $n_2 = 1.3$) coinciden con el ejemplo de fibra óptica del Capítulo 4 del reporte (“núcleo de alta densidad $n_1 = 1.5$... revestimiento... $n_2 = 1.3$ ”), lo cual conecta directamente con la Pestaña 3 (Guía de Onda) descrita más abajo.

```

1 # --- CÁLCULOS MATEMÁTICOS BÁSICOS ---
2 theta_t_deg, tir, theta_critico = calcular_snell(n1, n2, theta_i_deg)
3
4 x_A, y_A = -4.0, 4.0
5 x_B, y_B = 4.0, -4.0
6

```

```

7 opl_puntual, x_array, opl_array, x_min, opl_min = analizar_camino_optico(
8     n1, n2, x_A, y_A, x_B, y_B, x_interfaz
9 )

```

Explicación

Llama a las funciones del motor matemático descritas en la Sección 2: primero resuelve Snell/TIR para el ángulo actual; después fija los puntos fuente y detector $A(-4, 4)$ y $B(4, -4)$ (medio 1 arriba, medio 2 abajo) y calcula el Camino Óptico completo para la posición actual del deslizador de Fermat.

Vínculo con el reporte

Los puntos A y B fijados aquí, $A(-4, 4)$ y $B(4, -4)$, son un caso particular de la parametrización general $A(x_A, y_A)$, $B(x_B, y_B)$ usada en el cuadernillo del Módulo I para el Ejercicio 3 (Comprobación Variacional).

```

1 with col1:
2     st.subheader("Fundamento Teórico")
3     st.markdown("La Ley de Snell describe la refracción geométrica en la frontera:")
4     st.latex(r"n_1 \sin(\theta_i) = n_2 \sin(\theta_t)")
5
6     if tir:
7         st.error(f"**Reflexión Total Interna (TIR)**\nEl ángulo crítico es {theta_critico:.2f}°.")
8     else:
9         st.success(f"**Ángulo de Refracción:** {theta_t_deg:.2f}°")
10        if theta_critico:
11            st.info(f"Ángulo crítico del sistema: {theta_critico:.2f}°")

```

Explicación

Muestra la fórmula de Snell en LaTeX y, condicionalmente: si hay TIR ($\text{tir}=\text{True}$), despliega un recuadro rojo de error con el ángulo crítico; si no, un recuadro verde con el ángulo refractado, y opcionalmente (si existe ángulo crítico para esta combinación de índices) un recuadro azul informativo con su valor, aunque todavía no se haya alcanzado.

```

1     st.divider()
2     st.subheader("Análisis de Energía (Fresnel)")
3     Rs, Rp, R_main, Ts, Tp, T_main = calcular_fresnel(n1, n2, theta_i_deg)
4
5     st.markdown("**Luz no polarizada (Promedio):**")
6     met1, met2 = st.columns(2)
7     met1.metric(label="Reflectancia (R)", value=f"{R_main*100:.1f} %")
8     met2.metric(label="Transmitancia (T)", value=f"{T_main*100:.1f} %")
9
10    st.markdown("**Componentes de Polarización:**")
11    c1, c2 = st.columns(2)
12    c1.metric("R_s", f"{Rs*100:.1f} %")
13    c2.metric("R_p", f"{Rp*100:.1f} %")
14
15    c3, c4 = st.columns(2)
16    c3.metric("T_s", f"{Ts*100:.1f} %")
17    c4.metric("T_p", f"{Tp*100:.1f} %")
18
19    if Rp < 0.001 and not tir and theta_i_deg > 0:
20        st.info("[i] **;Ángulo de Brewster detectado!** La componente paralela ($p$) se transmite por completo y la luz reflejada queda 100% polarizada en $s$.")

```


Explicación

Llama a `calcular_fresnel` y despliega seis métricas numéricas en un arreglo de columnas: reflectancia y transmitancia promedio (luz no polarizada), y las cuatro componentes individuales R_s , R_p , T_s , T_p . La última condición (**L.19**) detecta el ángulo de Brewster *de forma numérica*: cuando R_p cae por debajo de un umbral casi nulo (y no hay TIR ni incidencia normal), se activa una alerta didáctica.

Vínculo con el reporte

La detección del ángulo de Brewster es una funcionalidad real y activa del simulador que **no está descrita en el cuerpo del reporte** (solo aparece en el capítulo de verificación numérica, actualmente comentado, en el Caso 1 del Módulo II). Es una oportunidad para enriquecer la sección activa del reporte, ya que el código ya la implementa y funciona.

```

1  st.divider()
2  st.subheader("Principio de Fermat")
3  st.markdown("Establece que la luz sigue la trayectoria que minimiza el tiempo de viaje o **
4  Camino Óptico (OPL)**:")
5  st.latex(r"OPL(x) = n_1 L_1 + n_2 L_2")
6
7  st.metric(label="Camino Óptico Actual", value=f"{opl_puntual:.4f}")
8  st.metric(label="Mínimo Absoluto (Fermat)", value=f"{opl_min:.4f}")
9
10 if abs(x_interfaz - x_min) < 0.15:
11     st.success(f"¡Trayectoria Real!  $x \approx \{x_{\min}:.2f\}$  cm -- "
12               f"este punto minimiza el OPL y cumple la Ley de Snell.")
13 else:
14     diferencia = opl_puntual - opl_min
15     st.warning(f"Trayectoria de prueba -- el OPL actual excede al mínimo "
16               f"en  $\{diferencia:.4f\}$  unidades. "
17               f"Mueva el deslizador hacia  $x \approx \{x_{\min}:.2f\}$  cm para "
18               f"encontrar la trayectoria que la naturaleza elige.")

```

Explicación

Muestra el valor actual del OPL y su mínimo absoluto como métricas numéricas, y da retroalimentación inmediata al estudiante: si el punto de cruce elegido está suficientemente cerca del mínimo real ($|x_{\text{interfaz}} - x_{\min}| < 0.15$), confirma que encontró la “trayectoria real”; si no, informa cuánto excede el OPL actual al mínimo y sugiere hacia dónde mover el deslizador.

Vínculo con el reporte

Esta retroalimentación interactiva es la implementación directa del ciclo POE del cuadernillo del Módulo I, Ejercicio 3: el estudiante desliza x hasta que el recuadro indique “¡Trayectoria Real!”, y después verifica los ángulos con la Ley de Snell — exactamente el flujo que describe la Fase 2 de ese ejercicio.

```

1  with col2:
2      tab1, tab2, tab3 = st.tabs(["1. Trazado de Rayos (Snell)", "2. Minimización (Fermat)", "3. Guía de Onda (Fibra Óptica)"])

```

Explicación

Crea las tres pestañas de la columna derecha. El resto del archivo (`with tab1:`, `with tab2:`, `with tab3:`) dibuja el contenido de cada una por separado; se documentan a continuación de forma resumida, ya que su contenido es principalmente de graficación (`matplotlib`) y no introduce física nueva más allá de la ya calculada arriba.

```

1  with tab1:
2      # ... Construye la figura de matplotlib con:
3      # - la interfaz horizontal y los "fondos" de ambos medios (fill_between)
4      # - el rayo incidente (rojo), calculado desde theta_i_deg
5      # - el rayo reflejado (naranja), con grosor y opacidad proporcionales a R_main
6      # - el rayo refractado (verde, si no hay TIR), con grosor y opacidad
7      #   proporcionales a T_main
8      # - arcos y etiquetas para theta_i, theta_r, theta_t
9      st.pyplot(fig_snell)

```

Explicación

`grosor_ref = max(1.0, 4.0 * R_main)` y `alpha_ref = max(0.3, R_main)` son las líneas clave: convierten el valor numérico de la reflectancia/transmitancia en un atributo visual (grosor de línea y opacidad) del rayo correspondiente, de modo que un vistazo a la figura ya comunica cuánta energía se refleja y cuánta se transmite.

Vínculo con el reporte

Es la implementación exacta de la descripción del reporte: *“el simulador traduce los coeficientes de reflectancia (R) y transmitancia (T) en parámetros visuales, modulando el grosor y la opacidad (canal alfa) de los vectores renderizados por matplotlib”* (Capítulo 4, Módulo I).

```

1  with tab2:
2      # Columna izquierda: geometría del rayo de prueba (A -> x_interfaz -> B)
3      #   con arcos de theta_i y theta_t medidos respecto a la normal.
4      # Columna derecha: gráfica de OPL(x) contra x, con un punto rojo en la
5      #   posición actual del deslizador y una línea vertical verde en el
6      #   mínimo teórico x_min.
7      st.pyplot(fig_geom)
8      st.pyplot(fig_fermat)

```

Explicación

Dibuja, lado a lado, la geometría del rayo de prueba (usando los mismos puntos A y B fijos) y la curva completa del Camino Óptico como función de x , marcando tanto la posición actual elegida por el estudiante como el mínimo real.

Vínculo con el reporte

Esta gráfica de `fig_fermat` es, literalmente, el “pozo de potencial” que describe el reporte: *“demostrando gráficamente que la trayectoria real de la luz coincide invariablemente con el mínimo absoluto del pozo de potencial generado por la curva LCO”* — aunque, como se señaló antes, el eje de esta gráfica está etiquetado “Camino Óptico Relativo” y las métricas dicen OPL, no LCO.

```

1  with tab3:
2      if n1 <= n2:
3          st.warning("[!] **Pérdida de señal:** Para que exista confinamiento total, el índice
4              del núcleo (Medio 1) debe ser estrictamente mayor al índice del revestimiento (
5              Medio 2).")
6
7      # Ciclo for con max_rebotes = 15:
8      #   en cada iteración calcula la Ley de Snell con el ángulo fijo theta_i_deg;
9      #   si hay TIR, el rayo rebota (cambia dir_y de signo) y continúa dentro
10     # del núcleo; si no hay TIR, el rayo escapa (se dibuja una fuga en rojo
11     #   punteado) y el ciclo termina con "break".
12     st.pyplot(fig_fibra)

```

Explicación

Simula el confinamiento de un rayo dentro de una guía de onda: en cada rebote contra la frontera núcleo/revestimiento evalúa si ocurre TIR (rebote, la señal se queda confinada) o no (fuga, la señal escapa y se dibuja el rayo perdido en rojo punteado, terminando el ciclo con `break`). El aviso de la línea 2 protege contra la configuración físicamente imposible ($n_1 \leq n_2$, sin confinamiento posible).

Vínculo con el reporte

Implementa directamente el Caso 4 del capítulo de verificación numérica del reporte (actualmente comentado): *“Régimen de confinamiento (TIR): ... el motor detecta la condición de Reflexión Total Interna. El algoritmo iterativo preserva la trayectoria en el medio n_1 ...”* — de nuevo, una funcionalidad real y activa que el cuerpo principal del reporte no describe fuera del capítulo comentado.

4 2_Módulo_II_Arreglo_de_Lentes.py

```

1 import streamlit as st
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from utils_math import trazar_rayo_sistema, obtener_matriz_sistema
5
6 def dibujar_flecha(ax, z, y, tipo="objeto"):
7     """Renderiza la flecha del objeto o la imagen respetando convenciones ópticas."""
8     if tipo == "objeto":
9         color = "green"; alfa = 1.0; estilo = "-"; etiqueta = "Objeto"
10    elif tipo == "real":
11        color = "darkorange"; alfa = 1.0; estilo = "-"; etiqueta = "Img. Real"
12    elif tipo == "virtual":
13        color = "purple"; alfa = 0.6; estilo = "--"; etiqueta = "Img. Virtual"
14
15    ax.annotate('', xy=(z, y), xytext=(z, 0),
16               arrowprops=dict(arrowstyle="->", color=color, linewidth=3, alpha=alfa, ls=estilo))
17
18    desplazamiento = 0.8 if y > 0 else -0.8
19    ax.text(z, y + desplazamiento, etiqueta, color=color, fontsize=10,
20           fontweight='bold', ha='center', va='center', alpha=alfa)

```

Explicación

Función auxiliar de dibujo reutilizada tres veces (objeto, imagen real, imagen virtual): asigna un color y estilo de línea distintos a cada tipo (verde sólido para el objeto, naranja sólido para imagen real, morado punteado y semitransparente para imagen virtual — codificando visualmente que “no es luz físicamente presente”), y dibuja una flecha vertical desde el eje óptico hasta la altura y usando `ax.annotate`, con la etiqueta de texto colocada arriba o abajo de la punta según el signo de y .

Vínculo con el reporte

La convención de color/estilo (línea continua vs. punteada semitransparente) para distinguir imágenes reales de virtuales es la representación visual del contenido teórico del reporte sobre formación de imágenes en el formalismo ABCD (Capítulo 4, Módulo II), aunque el reporte no especifica una convención de color en particular — es una decisión de diseño de la interfaz.

```

1 st.set_page_config(page_title="Módulo I: Óptica Geométrica", layout="wide")
2 st.title("Módulo I: Formación de Imágenes y Trazado Paraxial")
3 st.markdown("Banco óptico virtual para el diseño de sistemas formadores de imágenes.")
4
5 if 'componentes' not in st.session_state:
6     st.session_state.componentes = []

```

Explicación

`st.session_state` es el mecanismo de Streamlit para conservar datos entre interacciones del usuario (que de otro modo se perderían, ya que Streamlit re-ejecuta todo el script en cada clic). Aquí se usa para mantener la lista de lentes que el estudiante ha ido agregando al banco óptico.

Nota de congruencia

Igual que en el archivo anterior: el título en pantalla dice “**Módulo I**”, mientras que en el reporte y los cuadernillos este contenido (formalismo ABCD, formación de imágenes) corresponde al “**Módulo II**”. Es el mismo par de líneas que hay que intercambiar con las del archivo de Reflexión y Refracción para que ambos títulos coincidan con el reporte.

```

1 with st.sidebar.form("form_fuente"):
2     modo = st.radio("Modo de Emisión", ["Formación de Imagen (Objeto)", "Rayo Único"])
3
4     if modo == "Rayo Único":
5         y0 = st.slider("Altura inicial  $y_0$  (cm)", -5.0, 5.0, 1.0, 0.1)
6         theta0 = st.slider("Ángulo inicial  $\theta_0$  (°)", -15.0, 15.0, 0.0, 0.5)
7     else:
8         y0 = st.slider("Altura del Objeto (cm)", 0.5, 5.0, 2.5, 0.1)
9     ...
10    btn_calcular = st.form_submit_button("Calcular / Actualizar")

```

Explicación

Ofrece dos modos de emisión: **Rayo Único** (un solo rayo con altura y ángulo iniciales arbitrarios — corresponde al Ejercicio 1 del cuadernillo, el telescopio de Kepler) y **Formación de Imagen** (un objeto extendido, del cual se trazan varios rayos para localizar la imagen — corresponde al Ejercicio 2, sobre el mito de los “rayos especiales”). Usar `st.form` agrupa varios controles para que el cálculo solo se dispare al presionar el botón, en vez de recalcular en cada movimiento de deslizador.

```

1 st.sidebar.header("2. Agregar Componente Óptico")
2 with st.sidebar.form("form_agregar"):
3     focal_input = st.number_input("Distancia Focal  $f$  (cm)", value=10.0, step=1.0)
4     distancia_input = st.number_input("Distancia desde el elemento anterior (cm)", value=15.0,
5                                     min_value=0.0, step=1.0)
6
7     btn_agregar = st.form_submit_button("Añadir Lente al Sistema")
8     if btn_agregar:
9         st.session_state.componentes.append(('lente', focal_input, distancia_input))
10        st.rerun()

```

Explicación

Formulario para agregar un nuevo componente óptico a la lista: cada vez que se presiona el botón, agrega una tupla ('lente', f , d) a `st.session_state.componentes` y fuerza un re-renderizado (`st.rerun()`) para reflejar el cambio de inmediato.

Nota de congruencia

Este formulario solo construye componentes de tipo 'lente'. Aunque `utils_math.py` sí implementa `matriz_espejo_esferico` y ambas funciones de trazado (`trazar_rayo_sistema`, `obtener_matriz_sistema`) manejan explícitamente la rama `tipo == 'espejo'`, no existe ningún control en esta interfaz para que el usuario agregue un espejo al banco óptico. La matriz de espejo del Apéndice D del reporte queda, en la práctica, inalcanzable desde la aplicación.

```

1 st.subheader("Análisis Matriz ABCD")
2 M_sys = obtener_matriz_sistema(st.session_state.componentes)
3 A, B, C, D = M_sys[0,0], M_sys[0,1], M_sys[1,0], M_sys[1,1]
4
5 st.latex(r"M_{sys} = \begin{pmatrix} " + f"{A:.4f} & {B:.4f} \\\ \" + r"{C:.4f} & {D:.4f} " + r"\end{pmatrix}")
6
7 if C != 0:
8     f_eq = -1.0 / C
9     h1 = (D - 1.0) / C
10    h2 = (A - 1.0) / C
11
12    st.metric(label="Distancia Focal Equivalente ( $f_{eq}$ )", value=f"{f_eq:.2f} cm")
13    with st.expander("Ver Planos Principales"):
14        st.write(f"● **Plano Principal Principal ( $h_1$ ):** {h1:.2f} cm (desde el inicio)")

```

```

15         st.write(f"● **Plano Principal de Salida (${h_2$}):** {h2:.2f} cm (desde la última lente)")
16     else:
17         st.warning("Sistema Afocal (Focal equivalente infinita). Las lentes están configuradas como un telescopio colimador.")

```

Explicación

Obtiene M_{sys} con `obtener_matriz_sistema` y extrae sus cuatro elementos A, B, C, D ; los muestra como matriz en LaTeX. Si $C \neq 0$: calcula $f_{eq} = -1/C$ y dos posiciones de plano principal (h_1 desde D , h_2 desde A), mostradas dentro de un `expander` colapsable. Si $C = 0$: en vez de dividir entre cero, se despliega la alerta de sistema afocal.

Vínculo con el reporte

$f_{eq} = -1/C$ es la ecuación del reporte. La detección de sistema afocal cuando $C = 0$ corresponde a la definición del reporte: “cuando $C = 0$, el arreglo se considera un sistema afocal... donde los rayos que inciden paralelos al eje óptico emergen igualmente paralelos a la salida”.

Nota de congruencia

$h_2 = (A - 1.0) / C$ en el código corresponde algebraicamente a $\delta_{H'} = \frac{A-1}{C} = f_{eq} - \text{DFP}$. Sin embargo, la ecuación del reporte está escrita como $\delta_{H'} = \frac{1-A}{C}$ (signo opuesto al del código). Vale la pena revisar cuál de las dos versiones es la correcta y corregir la que esté mal, ya que actualmente el reporte y el código no coinciden en el signo de esta cantidad. Además, $h_1 = (D - 1.0) / C$ (etiquetado “Plano Principal Principal”) usa una fórmula análoga con D en vez de A , para la cual el reporte no ofrece ninguna derivación explícita.

```

1  if modo == "Formación de Imagen (Objeto)" and len(rayos_trazados) >= 2:
2      z_rayo1, y_rayo1 = rayos_trazados[0]
3      z_rayo2, y_rayo2 = rayos_trazados[1]
4
5      z_a, z_b = z_rayo1[-2], z_rayo1[-1]
6      y1_a, y1_b = y_rayo1[-2], y_rayo1[-1]
7      y2_a, y2_b = y_rayo2[-2], y_rayo2[-1]
8
9      dz = z_b - z_a
10     if dz > 0:
11         m1 = (y1_b - y1_a) / dz
12         m2 = (y2_b - y2_a) / dz
13
14         if abs(m1 - m2) > 1e-5:
15             c1 = y1_b - m1 * z_b
16             c2 = y2_b - m2 * z_b
17
18             z_imagen = (c2 - c1) / (m1 - m2)
19             y_imagen = m1 * z_imagen + c1
20
21             if z_imagen < z_acumulado:
22                 # ... dibuja líneas de extensión virtual (punteadas)
23                 dibujar_flecha(ax, z_imagen, y_imagen, tipo="virtual")
24             else:
25                 dibujar_flecha(ax, z_imagen, y_imagen, tipo="real")

```

Explicación

Este es el bloque que localiza la posición de la imagen *geoméricamente*, sin usar la ecuación de lentes delgadas: toma los dos últimos puntos trazados de dos rayos distintos que salieron del mismo objeto (L.2–3), calcula la pendiente de cada segmento final (L.9–10, $m = \Delta y / \Delta z$) y encuentra la intersección de ambas rectas resolviendo el sistema lineal $y = m_1 z + c_1 = m_2 z + c_2$ (L.13–15). Si esa intersección ocurre *antes* de la última

lente ($z_{\text{imagen}} < z_{\text{acumulado}}$), es una imagen virtual (los rayos divergen realmente, y solo su *extensión hacia atrás* se cruza); si ocurre después, es una imagen real (los rayos convergen físicamente ahí).

Vínculo con el reporte

Esta es la implementación numérica exacta del objetivo pedagógico central del cuadernillo de Módulo I (Ejercicio 2): superar el “mito de los rayos especiales” verificando que la imagen se localiza correctamente usando *cualquier* par de rayos del haz (no solo los tres especiales), porque *“todo rayo contenido en ese haz... retro-proyecta exactamente al mismo punto virtual”*, tal como concluye la Solución 2 del cuadernillo.

5 3_Módulo_III_Interferencia.py

```

1 st.sidebar.header("Selección de Experimento")
2 tipo_experimento = st.sidebar.radio(
3     "Mecanismo de Interferencia",
4     ["Doble Rendija de Young (Div. Frente)", "Interferómetro de Michelson (Div. Amplitud)"]
5 )

```

Explicación

Control tipo `radio` que decide cuál de los dos experimentos se muestra; el resto del archivo está estructurado como un gran `if/else` sobre esta variable, replicando en código la distinción teórica del cuadernillo entre división de frente de onda y división de amplitud.

```

1 st.latex(r"\vphantom{\Bigg|} I(y) = 4I_0 \cos^2 \left( \frac{\Delta\phi}{2} \right)")
2 st.latex(r"\vphantom{\Bigg|} \Delta\phi = \frac{2\pi}{\lambda}(r_2 - r_1)")
3 st.info("**Nota:** Se calculan distancias vectoriales exactas en el espacio, no solo
4     aproximaciones paraxiales.")
5
6 X, Y, intensidad = calcular_patron_young(lam_nm, d_mm, L_cm, tamano_cm)

```

Explicación

`\vphantom{\Bigg|}` es un truco tipográfico de LaTeX: inserta un espacio vertical invisible del tamaño de un paréntesis grande, para que ambas ecuaciones (mostradas en columnas paralelas) queden alineadas verticalmente aunque una tenga fracciones más altas que la otra. La llamada a `calcular_patron_young` dispara el cálculo real descrito en la Sección 2.3 de este documento.

Vínculo con el reporte

Las dos ecuaciones desplegadas corresponden a $I = 4I_0 \cos^2(\delta/2)$ y $\delta = k(r_2 - r_1)$, ambas del reporte.

```

1 if lam_nm < 495: colormap, linecolor = 'Blues', 'blue'
2 elif lam_nm < 570: colormap, linecolor = 'Greens', 'green'
3 elif lam_nm < 620: colormap, linecolor = 'Oranges', 'orange'
4 else: colormap, linecolor = 'Reds', 'red'

```

Explicación

Traduce la longitud de onda numérica a un color aproximado del espectro visible (azul, verde, naranja, rojo) tanto para el mapa de color 2D como para la curva 1D, de modo que si el estudiante mueve el deslizador de λ hacia el azul, la gráfica completa cambia de paleta — refuerzo visual directo del Ejercicio 1 del cuadernillo (“dependencia paramétrica”).

```

1 fig_2d, ax_2d = plt.subplots(figsize=(6, 4.5))
2 cax = ax_2d.imshow(intensidad, extent=[-tamano_cm/2, tamano_cm/2, -tamano_cm/2, tamano_cm/2],
3     cmap=colormap, origin='lower', aspect='auto')
4
5 st.pyplot(fig_2d)
6
7 eje_y_cm = Y[:, 0] * 100.0
8 ax_1d.plot(eje_y_cm, intensidad[:, intensidad.shape[1] // 2], color=linecolor, linewidth=2)

```


Explicación

La gráfica 2D usa `imshow` para pintar la matriz completa de intensidad como una imagen (el patrón de franjas que vería el ojo sobre la pantalla). La gráfica 1D extrae una sola columna del centro de esa misma matriz (`intensidad[:, ... // 2]`) — es decir, un corte transversal por el eje central — y la grafica como curva de intensidad contra posición y (convertida de metros a centímetros multiplicando por 100).

Vínculo con el reporte

Esta disposición de gráficas “2D e 1D lado a lado” es exactamente lo que describe el reporte: *“La interfaz gráfica se estructuró en columnas paralelas, correlacionando instantáneamente el interferograma 2D con un perfil transversal de irradiancia unidimensional”*.

```
1 delta_y_paraxial_cm = ((lam_m * L_m) / d_m) * 100.0
2 y1_exacto_cm = np.sqrt((lam_m**2 * L_m**2) / (d_m**2 - lam_m**2) + (lam_m**2 / 4.0)) * 100.0
3 error_porcentual = abs(y1_exacto_cm - delta_y_paraxial_cm) / y1_exacto_cm * 100.0
4
5 col_metric1.metric(label="Predicción Paraxial ( $\Delta y$ )", value=f"{delta_y_paraxial_cm:.4f} cm")
6 col_metric2.metric(label="Posición Exacta (Simulador)", value=f"{y1_exacto_cm:.4f} cm")
```

Explicación

Calcula, en paralelo, la fórmula paraxial $\Delta y \approx \lambda L/d$ y la fórmula exacta hiperbólica $y_1 = \sqrt{\lambda^2 L^2 / (d^2 - \lambda^2) + \lambda^2 / 4}$, para que el estudiante compare ambas directamente. El `error_porcentual` se calcula pero **está comentado** en las líneas siguientes (`#with col_metric3: #... st.metric(...)`), por lo que actualmente no se muestra en pantalla, aunque el cálculo sí se ejecuta.

Vínculo con el reporte

La fórmula exacta es la ecuación final del Apéndice A del reporte, $y_m = \sqrt{\frac{m^2 \lambda^2 L^2}{d^2 - m^2 \lambda^2} + \frac{m^2 \lambda^2}{4}}$, evaluada aquí para el primer máximo ($m = 1$). Es la confirmación en código de que la derivación algebraica completa de ese apéndice sí se usa activamente en la aplicación, no es solo un ejercicio teórico aislado.

```
1 delta_d_um = st.sidebar.slider(
2     "Desplazamiento del Espejo M2 ( $\mu\text{m}$ )",
3     0.000, 50.000, 10.000, 0.001,
4     format="%.3f"
5 )
6 ...
7 st.latex(r"\Delta \phi = \frac{4\pi}{\lambda} \Delta d \cos(\theta)")
8 X, Y, intensidad = calcular_patron_michelson(lam_nm, delta_d_um, f_cm, tamano_cm)
```

Explicación

Control de desplazamiento del espejo en micrómetros, con resolución de una milésima de micra (`step=0.001`) — consistente con la altísima sensibilidad real del instrumento. La ecuación mostrada aquí sí incluye el factor $\cos \theta$ explícitamente.

Vínculo con el reporte

Esta ecuación desplegada en pantalla, $\Delta \phi = \frac{4\pi}{\lambda} \Delta d \cos \theta$, es la versión *correcta y completa* de la fase de Michelson — coincide con lo que realmente calcula `calcular_patron_michelson` (véase Sección 2.3) y con el pie de la figura del interferómetro de Michelson en el reporte, pero *no* con la ecuación del cuerpo del reporte, que omite el $\cos \theta$ (véase la Nota de congruencia en la Sección 2.3).

```
1  paso_lambda = (lam_nm * 1e-3) / 2.0  # en micras
2
3  centro_idx_y = intensidad.shape[0] // 2
4  centro_idx_x = intensidad.shape[1] // 2
5  intensidad_central = intensidad[centro_idx_y, centro_idx_x]
6
7  col_metric1.metric(label="Sensibilidad ( $\lambda/2$ )", value=f"{paso_lambda:.4f}  $\mu\text{m}$ ")
8  col_metric2.metric(label="Intensidad Relativa Central", value=f"{intensidad_central:.3f}")
```

Explicación

Calcula $\lambda/2$ en micras (convirtiendo primero λ de nm a micras con $1\text{e-}3$) y extrae el valor de intensidad exactamente en el píxel central de la malla (fila y columna a la mitad de la matriz), para mostrar numéricamente si el centro del patrón está en un máximo o mínimo brillante en ese instante.

Vínculo con el reporte

`paso_lambda` es la cantidad $\Delta d = \lambda/2$ del “Cuadro de Referencia Rápida” del cuadernillo del Módulo III (“Desplazamiento del espejo por franja”), y es exactamente lo que el Ejercicio 2 de ese cuadernillo pide predecir antes de revisar el simulador.

6 4_Módulo_IV_Difracción.py

```

1 geometria = st.sidebar.radio("Tipo de Apertura", ["Rendija Simple (1D)", "Apertura Rectangular (2D)"])
2 ...
3 if geometria == "Apertura Rectangular (2D)":
4     b_mm = st.sidebar.slider("Alto de la apertura 'b' (mm)", 0.01, 0.50, 0.15, 0.01)
5 else:
6     # Corrección: 0.0 anula el encogimiento en Y, haciendo que las franjas llenen la pantalla
7     b_mm = 0.0

```

Explicación

Cuando el modo es “Rendija Simple (1D)”, el control del alto b se oculta y se fuerza $b_{\text{mm}} = 0.0$. Este valor no representa una apertura de altura físicamente nula, sino un indicador interno para que la función de cálculo (véase más abajo) sepa que debe usar solo el patrón unidimensional, sin aplicar el factor sinc^2 en y .

```

1 escala = st.sidebar.radio("Escala de Intensidad", ["Lineal", "Logarítmica"],
2                             help="La escala logarítmica permite visualizar los máximos secundarios
3                                 de baja intensidad.")
4 ...
5 st.latex(r"\vphantom{\Big|} I(x,y) = I_0 \text{sinc}^2(\alpha) \text{sinc}^2(\beta)")
6 st.latex(r"\vphantom{\Big|} \alpha = \frac{\pi a x}{\lambda L}, \quad \beta = \frac{\pi b y}{\lambda L}")

```

Explicación

El control de escala (Lineal/Logarítmica) determina más adelante cómo se normaliza el mapa de color (`mcolors.LogNorm` vs. `mcolors.Normalize`). Las dos ecuaciones desplegadas presentan la forma general 2D del patrón de difracción y la definición *paraxial* de las variables auxiliares α, β (usando $\sin \theta \approx x/L$).

Vínculo con el reporte

La fórmula $I = I_0 \text{sinc}^2(\alpha) \text{sinc}^2(\beta)$ es la ecuación del reporte. Sin embargo, como se detalla en la Nota de congruencia de la Sección 2.4, la definición de α, β que se *muestra* aquí ($\alpha = \pi ax/(\lambda L)$, aproximación paraxial) no es exactamente la que *calcula* `calcular_difraccion_rendija/calcular_difraccion_rectangular` (que usan el seno geométrico exacto $X/\text{distancia}$).

```

1 if escala == "Logarítmica":
2     norm = mcolors.LogNorm(vmin=1e-3, vmax=1.0)
3     plot_data = np.clip(intensidad, 1e-4, 1.0)
4 else:
5     norm = mcolors.Normalize(vmin=0.0, vmax=0.4)
6     plot_data = intensidad

```

Explicación

En modo logarítmico, `np.clip` evita valores de intensidad exactamente cero (que no tienen logaritmo definido) sustituyéndolos por un piso de 10^{-4} ; `LogNorm` mapea después esos valores a la escala de color en $\log_{10}$. En modo lineal, el rango de color se satura deliberadamente en `vmax=0.4` (no en 1.0) para que los máximos secundarios — mucho más tenues que el central — sean visibles en vez de quedar invisibles junto al pico dominante.

Vínculo con el reporte

Esta es la implementación exacta del recurso pedagógico que describe el reporte: “la interfaz gráfica incorpora una transformación de visualización en escala logarítmica, la cual comprime el rango dinámico de los datos

y permite al estudiante inspeccionar visualmente la estructura de los lóbulos secundarios". El dato numérico que motiva esta necesidad, el 4.72% de intensidad del primer máximo secundario, se deriva formalmente en el Apéndice B del reporte.

```

1 ancho_central_cm = ((2 * lam_m * L_m) / a_m) * 100.0
2 x1_cm = ancho_central_cm / 2.0
3 posicion_esperada_pico_cm = 1.4303 * x1_cm
4
5 if posicion_esperada_pico_cm > (tamano_cm / 2.0):
6     col_metric3.metric(label="Irradiancia 1er Máx. Secundario", value="Ampliar ventana")
7 else:
8     mascara_region_secundaria = eje_x_cm > (x1_cm * 1.01)
9     if np.any(mascara_region_secundaria):
10         irradiancia_secundaria = np.max(perfil_real_1d[mascara_region_secundaria])
11     else:
12         irradiancia_secundaria = 0.0
13     col_metric3.metric(label="Irradiancia 1er Máx. Secundario", value=f"{irradiancia_secundaria:.4f} I0")

```

Explicación

Calcula el ancho del máximo central $W = 2\lambda L/a$ y la posición esperada del primer máximo secundario multiplicando por la constante 1.4303 (la raíz de la ecuación trascendental $\tan \alpha = \alpha$). Antes de buscar ese máximo en los datos reales, verifica si cabe dentro de la ventana de observación actual; si no cabe, muestra el texto “Ampliar ventana” en vez de un número engañoso. Si sí cabe, filtra el perfil 1D para quedarse solo con la región más allá del primer mínimo ($\text{eje_x_cm} > \text{x1_cm} * 1.01$) y toma el máximo de esa región — es decir, encuentra el primer lóbulo secundario *numéricamente*, sin asumir su valor teórico.

Vínculo con el reporte

La constante 1.4303 y el valor teórico de 4.72% que debería obtenerse provienen directamente de la derivación del Apéndice B del reporte: *“la primera raíz no nula... se ubica numéricamente en $\alpha_1 \approx 1.4303\pi$... la intensidad del primer máximo secundario es apenas del 4.72% respecto al pico central”*. Que el código busque este valor *numéricamente* en los datos simulados (en vez de solo mostrar la constante teórica) es una validación cruzada en vivo de esa derivación cada vez que el estudiante usa el simulador.

7 5_Módulo_V_Polarización.py

```

1 def jones_a_euler_latex(Jx, Jy):
2     """Convierte un vector de Jones complejo a notación de Euler en LaTeX"""
3     def formatear_componente(z):
4         amp = np.abs(z)
5         fase = np.angle(z)
6         if amp < 1e-4:
7             return "0"
8         if abs(fase) < 1e-4:
9             return f"{amp:.2f}"
10        return f"{amp:.2f} e^{{{fase:+.2f}i}}"
11    ...

```

Explicación

Este archivo define *su propia* copia local de `jones_a_euler_latex`, casi idéntica a la que ya existe en `utils_math.py` (Sección 2.5), pero **nunca se importa desde `utils_math`** — de hecho el `import` al inicio del archivo no incluye esta función. Es una duplicación de código: dos implementaciones ligeramente distintas de la misma idea (la de aquí usa `{fase:+.2f}` con signo explícito; la de `utils_math.py` usa `i {fase:.2f}` sin signo forzado y con el orden `i, fase` invertido respecto a `fase, i`). No afecta la física, pero sí es una inconsistencia de mantenimiento: si se corrige un detalle de formato en una copia, la otra queda desactualizada.

```

1 st.sidebar.header("1. Fuente de Luz")
2 estado_inicial = st.sidebar.selectbox("Estado de Polarización Inicial",
3     ['Lineal Horizontal', 'Lineal Vertical', 'Lineal +45°', 'Circular Derecha', 'Circular
4     Izquierda'])

```

Explicación

Menú desplegable con los cinco estados canónicos de entrada, cuyos vectores de Jones exactos construye `vector_jones_inicial` (documentada en la Sección 2.5).

```

1 with st.sidebar.form("form_elementos"):
2     tipo = st.selectbox("Tipo de Elemento", ["Polarizador Lineal", "Lámina Retardadora"])
3     angulo = st.number_input("Ángulo de inclinación (grados)", value=45.0, step=5.0, ...)
4
5     retardo = 0.0
6     if tipo == "Lámina Retardadora":
7         tipo_retardo = st.selectbox("Desfase (Retardo)", ["Lámina de Cuarto de Onda ( $\lambda/4$ )", "Lámina de Media Onda ( $\lambda/2$ )", "Personalizado"])
8         if tipo_retardo == "Lámina de Cuarto de Onda ( $\lambda/4$ )": retardo = 90.0
9         elif tipo_retardo == "Lámina de Media Onda ( $\lambda/2$ )": retardo = 180.0
10        else: retardo = st.number_input("Retardo personalizado en grados", value=45.0)
11
12    btn = st.form_submit_button("Añadir al tren óptico")
13    if btn:
14        st.session_state.elementos_polarizacion.append({
15            'tipo': tipo, 'angulo': angulo, 'retardo': retardo
16        })
17    st.rerun()

```

Explicación

Formulario para agregar un elemento óptico al “tren” (lista secuencial): si el usuario elige “Lámina Retardadora”, aparece un segundo menú con los dos casos comerciales estándar (traduciendo directamente $\lambda/4 \rightarrow 90^\circ$ y $\lambda/2 \rightarrow 180^\circ$ de retardo) o la opción de introducir cualquier retardo personalizado. Cada elemento se guarda como un diccionario `{'tipo', 'angulo', 'retardo'}` en la lista de sesión.

Vínculo con el reporte

Esta es la interfaz de usuario para las dos láminas comerciales descritas en el Anexo de Matrices de Jones del reporte: “*Lámina de cuarto de onda ($\lambda/4$): Introduce un retardo de $\Gamma = \pi/2$* ” y “*Lámina de media onda ($\lambda/2$): Introduce un retardo de $\Gamma = \pi$* ” — los valores 90° y 180° del código son exactamente $\pi/2$ y π expresados en grados.

```

1 E_final = vector_jones_inicial(estado_inicial)
2
3 for elem in st.session_state.elementos_polarizacion:
4     if elem['tipo'] == "Polarizador Lineal":
5         M_comp = matriz_polarizador_lineal(elem['angulo'])
6     else:
7         M_comp = matriz_retardador(elem['angulo'], elem['retardo'])
8
9     E_final = np.dot(M_comp, E_final)
10
11 Ex, Ey = E_final[0,0], E_final[1,0]
12 intensidad = np.abs(Ex)**2 + np.abs(Ey)**2

```

Explicación

Este es el corazón físico del módulo: parte del vector de Jones inicial y, para *cada* elemento del tren óptico en el orden en que fue agregado, construye su matriz (polarizador o retardador) y la multiplica *por la izquierda* contra el vector acumulado hasta ese momento ($E_{\text{final}} = \text{np.dot}(M_{\text{comp}}, E_{\text{final}})$). Al final, extrae las dos componentes complejas y calcula la irradiancia como la suma de los cuadrados de sus magnitudes.

Vínculo con el reporte

Implementa la composición secuencial de operadores que describe el reporte: “*la modificación del estado de polarización por elementos ópticos anisotrópicos se modela algebraicamente mediante operadores lineales... que actúan multiplicativamente por la izquierda sobre el vector de Jones de entrada*” (ecuación conceptual $\vec{J}_{\text{salida}} = M \vec{J}_{\text{entrada}}$, generalizada aquí a una cadena $M_n \cdots M_2 M_1 \vec{J}_0$ para múltiples elementos), y a la afirmación: “*el apilamiento secuencial de múltiples elementos... es resuelto numéricamente en **numpy** mediante la multiplicación ordenada de sus respectivas matrices complejas*”.

```

1 def dibujar_fotograma(t_anim):
2     fig, ax = plt.subplots(figsize=(6, 6))
3     Ex_t, Ey_t = curva_elipse_polarizacion(Ex, Ey)
4     ...
5     if intensidad < 1e-4:
6         ax.text(0, 0, "Extinción Total\n(Oscuridad)", ha='center', va='center', color='red', ...)
7     else:
8         ax.plot(Ex_t, Ey_t, color='purple', linewidth=2.5, alpha=0.3)
9
10        x_inst = np.abs(Ex) * np.cos(-t_anim + np.angle(Ex))
11        y_inst = np.abs(Ey) * np.cos(-t_anim + np.angle(Ey))
12
13        ax.plot(x_inst, y_inst, 'go', markersize=8)
14        ax.arrow(0, 0, x_inst, y_inst, color='green', ...)

```

Explicación

Dibuja un solo fotograma de la animación: primero traza la elipse completa (trayectoria de un periodo completo, en morado tenue) usando `curva_elipse_polarizacion`; después calcula la posición *instantánea* del vector de campo en el tiempo `t_anim` evaluando directamente $E_x(t) = |E_x| \cos(-t + \arg E_x)$ y $E_y(t) = |E_y| \cos(-t + \arg E_y)$ (la parte real de $E e^{-i\omega t}$, calculada aquí de forma explícita en vez de usar `np.real` de un número complejo), y dibuja un punto verde y una flecha desde el origen hasta esa posición. Si la intensidad resultante

es prácticamente nula (por ejemplo, dos polarizadores cruzados a 90°), en vez de intentar dibujar un punto en el origen se muestra el texto “Extinción Total”.

```
1  animar = st.button("> Animar Propagación del Campo")
2  grafica_placeholder = st.empty()
3
4  if animar and intensidad >= 1e-4:
5      fases_t = np.linspace(0, 4 * np.pi, 60)
6      for t in fases_t:
7          fig = dibujar_fotograma(t)
8          grafica_placeholder.pyplot(fig)
9          plt.close(fig)
10         time.sleep(0.04)
11     fig_final = dibujar_fotograma(0.0)
12     grafica_placeholder.pyplot(fig_final)
13     plt.close(fig_final)
14 else:
15     fig_estatica = dibujar_fotograma(0.0)
16     grafica_placeholder.pyplot(fig_estatica)
17     plt.close(fig_estatica)
```

Explicación

`st.empty()` crea un contenedor vacío cuyo contenido puede sobrescribirse repetidamente sin acumular gráficas nuevas en la página. Al presionar el botón de animar, se generan 60 fotogramas cubriendo 4π radianes (dos vueltas completas de la elipse), dibujando y sobrescribiendo el contenedor cada 0.04 segundos (`time.sleep`), y cerrando cada figura de `matplotlib` (`plt.close(fig)`) para liberar memoria. Al terminar, regresa al fotograma en $t = 0$. Si no se anima, simplemente se muestra un único fotograma estático.

Vínculo con el reporte

Esta animación es la implementación literal de la descripción del reporte: “una subrutina asíncrona renderiza una animación paramétrica en tiempo real. Esta animación traza la trayectoria del vector de campo eléctrico transversal, permitiendo al estudiante identificar visualmente la geometría y la quiralidad (sentido de giro) de la elipse de polarización” (Capítulo 4, Módulo V).

8 Tabla Resumen: Vínculos y Notas de Congruencia

A modo de referencia rápida, la siguiente tabla concentra todas las notas de congruencia señaladas a lo largo del documento.

Ubicación en el código	Nota de congruencia con el reporte
analizar_camino_optico (utils_math.py)	En código y la interfaz usan el nombre OPL ; el cuerpo del reporte usa LCO para la misma cantidad.
calcular_patron_michelson (utils_math.py)	En código se incluye correctamente el factor $\cos \theta$; la ecuación del cuerpo del reporte no lo incluye.
vector_jones_inicial (utils_math.py)	El párrafo teórico correspondiente (“Vectores de Jones”) está comentado en el reporte.
trazar_rayo_sistema (utils_math.py)	Usa el ángulo geométrico α , no el ángulo reducido $n\alpha$ del reporte (equivalentes solo porque todo el sistema está en aire, $n = 1$).
1_Módulo_I...py, st.title	El título en pantalla dice “Módulo II”; el reporte usa “Módulo I” para este contenido.
2_Módulo_II...py, st.title	El título en pantalla dice “Módulo I”; el reporte usa “Módulo II” para este contenido (inverso exacto del caso anterior).
2_Módulo_II...py, formulario de componentes	Solo permite agregar lentes; la matriz de espejo del Apéndice D no es alcanzable desde la interfaz.
2_Módulo_II...py, cálculo de h2	$h2 = (A - 1.0) / C$ no coincide en signo con la ecuación del reporte, $\delta_{H'} = (1 - A)/C$.
2_Módulo_II...py, cálculo de h1	Se muestra en pantalla una fórmula análoga usando D para el plano principal frontal, sin derivación explícita en el reporte.
4_Módulo_IV...py, ecuación mostrada	Muestra $\alpha = \pi ax/(\lambda L)$ (paraxial), pero el cálculo real usa el seno geométrico exacto.
4_Módulo_IV...py, fórmula 2D mostrada	Se muestra la fórmula 2D incluso en modo “Rendija Simple (1D)”, donde b no aplica.
5_Módulo_V...py, jones_a_euler_latex	Función duplicada de forma independiente en vez de importarse desde utils_math.py.
Pestaña de fibra óptica (1_Módulo_I...py)	Funcionalidad completa y activa, descrita solo en el capítulo de verificación numérica comentado del reporte.
Detección de ángulo de Brewster (1_Módulo_I...py)	Funcionalidad completa y activa, sin mención en el cuerpo activo del reporte.